



Test corpus: what we validate against, and what we deliberately do not

The parsers in this repo are only as trustworthy as the input they have met. This records where that input comes from, and one thing we decided **not** to use.

What we use

1. Reference archives from SngCli

`internal/sng/testdata/reference-sngcli-v0.3.0.sng` was produced by **SngCli v0.3.0** (MIT, `mdsitton/SngFileFormat`) from a song folder we wrote. It is the only fixture that checks the reader against something other than our own understanding of the format — the fixtures in `mask_test.go` and `read_test.go` are built by an encoder written from the same reading of the spec as the reader, and two components sharing one misreading agree with each other perfectly.

SngCli is installed at `%LOCALAPPDATA%\Programs\sngcli\win-x64\SngCli.exe`. To regenerate:

```
SngCli.exe encode -i <folder-of-song-folders> -o <out> --noStatusBar -t 1
SngCli.exe decode -i <folder-of-sng-files> -o <out> --noStatusBar -t 1
```

`.gitignore` excludes `*.sng` so nobody commits a song library, with an explicit exception for `internal/sng/testdata/`.

2. YARG itself, as the end-to-end oracle

YARG v0.15.0 is installed portable at `%LOCALAPPDATA%\Programs\YARG\YARG.exe` — the official Windows x64 zip, no installer, no elevation, deletable in one command. LGPL-3.0-or-later, free, and downloaded from YARG's own release page.

This is the gate that matters for the writer: put a repacked `.sng` in a songs folder, let YARG scan it, and confirm it appears with the right metadata and is playable. Nothing we can assert about our own output substitutes for the client accepting it.

(YARC publishes a `.sig` beside each release asset but no plain checksum, and no public key we could find, so the download is recorded by its SHA-256 rather than verified:

`a2eac765a190e34a8acbdba5b1351844704b427349d6d36b8cf1096f575055e41`.)

3. Charts we author

Hand-written `song.ini` and chart files exercising the messy cases: BOMs, Latin-1 bytes, duplicate keys, missing section headers, values containing `=`, free-text years, unknown keys, absurd numbers. These are in the unit tests.

What we deliberately do not use

YARG's Official Setlist

It would be the obvious corpus — real charts, real metadata, distributed by YARC themselves — and we are **not** using it.

`YARC-Official/Official-Setlist-Public` states plainly that the setlist is proprietary, that "musicians gave permission to use these songs in YARG specifically", that everything "**must** be used exclusively for YARG", and that users should not download it manually. Using it as the test corpus for a third-party song server is outside that permission — and serving it from one would be squarely outside it.

That is not a close call, and it matters more here than it would elsewhere: this project's own goal is to make charts distributable *separately* from music precisely to avoid copyright problems. A project with that goal cannot be casual about the provenance of its own test data.

The same caution applies to bulk-downloading community charts from Chorus Encore or similar: individual charters' licensing varies, and most community packages bundle commercial audio. Individual charts may be used where the charter's own licence permits it — that is a per-chart question, answered before the chart is added, not a bulk decision.

There IS a documented framework, and it reopens this

The [YARN submission guidelines](#) define exactly which music may be used with YARG, which is a better answer than "self-authored only":

- **Acceptable:** CC BY, CC BY-SA, CC BY-NC, CC BY-NC-SA and CC0, at version 3.0 or 4.0; plus the catalogue of accepted royalty-free labels (currently NCS).
- **Not acceptable:** the no-derivatives licences, CC BY-ND and CC BY-NC-ND — because *visual synchronisation is itself a derivative work*. That is a subtlety worth knowing: a chart is a derivative of the song, not merely a companion to it.
- **Share-alike propagates:** a chart of a CC BY-SA song is, by definition, released under the same licence as the song.

- The wiki points at a curated list of Creative Commons songs (the "YARN Planning - CC" tab of the YARN Charts Master Spreadsheet) as a starting point.

So a legitimately-licensed corpus is reachable: a CC BY or CC0 song, charted, is redistributable in full — audio included — and could live in this repository as a real end-to-end fixture rather than a generated one. That is a better long-term answer than the synthetic corpus, and the only reason it is not the current answer is that nobody has charted one yet.

What the oracle actually found, 2026-09-05

`cmd/mkcorpus` wrote 20 deliberately awkward song folders (22 as of the UltraStar work); YARG v0.15.0 scanned them; its `badsongs.txt` and song cache were read back. **Our scanner had passed all 20 and was wrong about three of them.** These are the only findings in this repo that came from an oracle rather than from our own reasoning, which is precisely why they are the ones worth having.

As of the last run, every song YARG rejects is one this scanner independently flags, by three different routes: no parts detected (`13-mid-beats-chart`), `no_audio` (`19-no-audio`), and `ultrastar_no_title` (`21-ultrastar-no-title`). No divergences remain in the corpus.

Case	YARG	We had	Fixed
<code>song.ini</code> with no <code>[Song]</code> header	reads nothing , titles the song after its folder	read the keys	yes — parser now requires the header, and the song is flagged <code>no_metadata_section</code>
chart with no audio	rejects: <i>"No audio accompanying the chart file"</i>	accepted it silently	yes — flagged <code>no_audio</code>
folder with a chart but no <code>song.ini</code>	neither cached nor reported bad — silently skipped	accepted it silently	yes — flagged <code>no_song_ini</code>

The headerless case is the one that mattered most. Our leniency was a guess written into a comment ("some charts omit it"), and it would have produced a catalog whose titles disagreed with what the player sees on their own screen — a bug with no error message anywhere.

Confirmations, where YARG agreed with decisions we had reached from source:

- Duplicate keys resolve **last-wins** — `FIRST` never appears in YARG's cache, `SECOND` does. That is now three independent sources: `IniModifierCollection.cs`, SngCli's round-trip, YARG.
- Latin-1, UTF-16LE and UTF-8-BOM `song.ini` files all read correctly on both sides.
- `notes.mid` is a **hard selection** over `notes.chart`, not a preference. Given both, YARG chose the `.mid`, found it unplayable, and rejected the whole song rather than falling back. Hashing the wrong file would be a real divergence, exactly as `chartfile.go` warns.

- Uppercase `[SONG]` , ragged whitespace, CRLF, values containing `=` , free-text years, unknown keys, the `cover` override, clean/explicit stem variants and a 4-way drum kit all matched.

Both divergences are now closed

1. ~~**UltraStar** `notes.txt` .~~ **CLOSED**. YARG rejected it with *"Name metadata not provided"* despite `song.ini` carrying a name, and the reason turned out to be structural: **UltraStar is the one format whose metadata does not come from `song.ini`** . YARG reads `#TITLE` , `#ARTIST` and `#ALBUM` from the `.txt` itself and refuses the song outright when `#TITLE` is missing. It is also **vocals-only** — no instrumental part is derived from it, and harmony is keyed on `#PARTS:2` .

Re-measured with corrected fixtures: a valid UltraStar chart is accepted and **YARG titles it from the `.txt` , not from `song.ini`** — the corpus case deliberately carries a different name in each, and `song.ini` 's never appears in YARG's cache. Our scanner now does the same, reports lead vocals (plus harmony on `#PARTS:2`), and flags a titleless chart as `ultrastar_no_title` .

1. ~~**We do not know whether a chart is playable.**~~ **CLOSED** by the chart preparers. YARG rejected the header-only `.mid` with *"No notes found"*; our preparer now reports zero parts for that same chart, reaching the same conclusion by an independent route. A song with no detectable parts is still catalogued rather than dropped, but it no longer claims instruments it does not have.

The writer's gate, 2026-09-05

The same oracle was then pointed at output we produced. `yarg-song-server pack` repacked 14 corpus folders plus the reference song; YARG scanned the 15 archives and accepted 14, reading every title out of our metadata section.

The single rejection — `No notes found` — was settled by a **control** rather than explained away: SngCli's own archive of the same source folder was placed beside ours and scanned in the same pass, and YARG rejected both with the identical error. The fixture's hand-written chart has no note events. Without that control, "YARG rejected one of ours" would have looked like a writer bug, and chasing it would have been wasted work.

Independently, `SngCli decode` extracted our archive with zero errors and every contained file came back byte-identical to the source folder.

Reproducing this

```
go run ./cmd/mkcorpus -out $env:USERPROFILE\yarg-test\corpus
# point YARG at it by editing SongFolders in
# %USERPROFILE%\AppData\LocalLow\YARC\YARG
# release\settings.json
# then delete songcache.bin, launch YARG, and read badsongs.txt beside it
```

The corpus is generated, never committed: it is large, and a committed corpus rots into something nobody regenerates.

The gap this leaves, stated honestly

Self-authored input cannot produce unknown unknowns. We can write a `song.ini` with the mistakes we thought of; we cannot write one with the mistakes we did not. Two things partly close that gap:

- SngCli's output already taught us two things synthetic input never would — that duplicate keys resolve last-wins in the reference tool, and that a contained file's extension can lie about its container.
- YARG scanning our output is a real oracle, because it is the actual client with the actual parser.

Neither is a substitute for a large real library. If a legitimately-licensed corpus becomes available, revisit this.

That said, the first run of this method found three real bugs, two of which had been written into the code as confident comments. The oracle is doing work.

Second oracle run — 2026-09-05, against what the SERVER produced

The first run scanned loose folders. This one scanned the archives the running server handed a client, which is the thing Phase 2 actually promises: *an unmodified YARG reads what the server serves*. The two runs answer different questions and both are worth keeping.

Procedure, end to end and all of it measured:

```
go run ./cmd/mkcorpus -out $env:USERPROFILE\yarg-test\corpus      # 22 cases
go build -o yss.exe ./cmd/yarg-song-server
.\yss.exe -listen 127.0.0.1:8099 -songs $env:USERPROFILE\yarg-test\corpus -data .\data
# POST /api/v1/have with an empty list -> 22 missing
# GET /song/{hash}.sng for each -> 22 files
# point YARG's SongFolders at the downloaded folder, delete songcache.bin, launch, wait
```

Results:

Step	Outcome
Index the corpus	22 songs, 22 distinct charts, 0 problems, 15 ms
POST /have from an empty client	library_total=22 missing_count=22
Download every missing song	22 downloaded, 0 ambiguous, 0 failed
Re-scan the downloads with our own scanner	22 songs, 0 failures
YARG scans the 22 served archives	20 accepted, 2 refused

The two refusals, and our own independent verdict on the same archive:

Archive	YARG said	Our scanner said
30334065... "Mid Beats Chart"	No notes found	nothing — no issue at all
4ec68053... "No Audio"	No audio accompanying the chart file	issue no_audio, no stems

This section originally claimed the standard held, and it did not. The row above used to read " *parts_derived: true*, no part carries any difficulty" — which is an observation somebody made by reading the JSON, not an issue the scanner raised. The scanner emitted **no issue whatsoever** for 30334065... . Reading a field by eye and recording it as though the software had flagged it turned a miss into a pass, and the conclusion drawn from it — "the standard holds" — was false for one of the two songs it was about. Corrected on the third run below, where the gap was found and closed. The lesson is narrower than "check your work": **a green must come from the thing being tested, not from the tester noticing something.**

What this run turned up: packing gives an untitled UltraStar chart a title

Corpus case 21-ultrastar-no-title has a notes.txt with no #TITLE and a song.ini that does carry a name. As a **loose folder** YARG refuses it — UltraStar is the one format that takes its title from the chart rather than from song.ini, and a missing #TITLE is fatal ("Name metadata not provided"). Our scanner flags it ultrastar_no_title.

As a .sng produced by this server, YARG accepted it. That is not a packing bug: in a .sng the metadata lives in the archive's header section, and PackDir fills that section from song.ini, so the song genuinely has a title — scanning the archive shows its name resolved to "Has A Name In song.ini".

Correction, measured 2026-09-05 on the third run: this paragraph used to end "Our scanner reports no issue for the archive, which agrees with the client in both forms." It does not. Serving the packed archives and reading `/api/v1/songs` returns `ultrastar_no_title` on the archive as well, because the `notes.txt` inside it still has no `#TITLE`. That is a caveat on a song YARG plays, not a claim that YARG refuses it, and the issue text now says so in both directions. Worth keeping rather than suppressing: the chart really is missing a tag, and a player repacking it elsewhere would meet the folder behaviour again.

It is worth writing down anyway, because it means one sentence people will reasonably assume is not quite true: **the server does not always serve exactly what the folder was**. For this one format, repacking makes a song playable that was not. Nothing to fix; something to know before the Phase 2b sync client starts writing archives into a player's songs folder and a song appears that the player could not previously play.

Third oracle run — 2026-09-05, against a folder `yarg-sync` filled

The first run scanned loose folders. The second scanned archives downloaded by hand from the running server. This one is the Phase 2b exit criterion itself: **the real sync client filled an ordinary songs folder, and an unmodified YARG was pointed at it**. Nothing between the server and the game was done by hand.

```
.\\bin\\yarg-song-server.exe -songs $env:USERPROFILE\\yarg-test\\corpus -data .\\data -listen 127.0.0.1:8099
.\\bin\\yarg-sync.exe -server http://127.0.0.1:8099 -songs $env:USERPROFILE\\yarg-test\\phase2b\\syncd
# point YARG's SongFolders at that folder, delete songcache.bin, launch, wait ~45s,
# then read badsongs.txt beside it
```

Step	Outcome
Index the corpus	22 songs, 22 distinct charts, 0 problems
<code>yarg-sync</code> from an empty folder	22 downloaded, 0 failed, 229,515 bytes in 505 ms
Every archive verified by re-derived identity	22 / 22
YARG scans the synced folder	20 accepted, 2 refused

The refusals were the same two, for the same reasons, at the paths the client wrote:

```
C:\\Users\\ENG2\\yarg-test\\phase2b\\syncd\\30334065...sng    No notes found
C:\\Users\\ENG2\\yarg-test\\phase2b\\syncd\\4ec68053...sng    No audio accompanying the chart file
```

So the Phase 2b claim is now measured rather than asserted: an unmodified YARG plays from a folder this client filled, and it does so with no change to the game.

The fifth thing the oracle found, and the fourth defect

`13-mid-beats-chart` is a `notes.mid` carrying a beat track and no note tracks. YARG refuses it — `No notes found`. **Our scanner reported it clean.** Not "flagged with a different code": no issue at all. That breaks the standard this project exists to hold, and it had been recorded as a pass on the previous run (see the correction above).

Chasing it turned up a second, deeper defect. Two corpus cases are byte-identical apart from one line:

	20-ultrastar	21-ultrastar-no-title
<code>#TITLE</code> line	present	absent
every other byte	identical	identical
difficulties we derived	8	0

`PreparseUltraStar` returned early without deriving any part when `#TITLE` was missing, on the belief — written into the code and into a test — that YARG refuses such a chart outright, so its contents did not matter. **The second oracle run had already disproved that belief** (packed, the song plays) without anyone noticing that a function elsewhere still depended on it. So the server reported a song with no playable parts, for a song that plays.

That also made the obvious fix for the first defect wrong. A blanket "zero difficulties means no notes" rule would have flagged `21-ultrastar-no-title`, which YARG accepts — a false positive created by building on a known-bad number.

Both are fixed:

- `PreparseUltraStar` derives notes regardless of the title. What a chart *contains* does not depend on whether it is titled. The missing tag is recorded as a note, and the title question stays where it belongs, in the metadata layer.
- A new issue, `no_notes`, is raised when a chart parses cleanly and no instrument carries any difficulty. It is guarded on `PartsDerived`, because zero difficulties on a chart we *failed* to parse means "not determined" — the catalog already documents that distinction on the field itself, and claiming "no notes" there would report a conclusion never reached.
- `ultrastar_no_title`'s text asserted that YARG "refuses the song". It now states both measured behaviours: refused as a loose folder, played once packed.

Each change was red-proofed before its green was believed. Removing the `no_notes` check failed `TestNoNotesIsFlaggedBecauseYARGRejectsIt` and nothing else; dropping the `PartsDerived` guard failed `TestNoNotesIsNotClaimedWhenPartsWereNeverDerived` and nothing else; restoring the UltraStar early return failed both UltraStar tests and nothing else.

After the fix, the corpus reads:

Corpus case	Difficulties	Issues	YARG
13-mid-beats-chart	0	no_notes	refused
19-no-audio	8	no_audio	refused
21-ultrastar-no-title	8 (was 0)	ultrastar_no_title	accepted
the other 19	8 or 16	none, or a metadata caveat	accepted

The two songs YARG refuses are exactly the two this scanner flags as refusals. Song identity was re-checked and is unchanged by all of this — `SHA1(chart bytes)` does not move because our derivation improved.

What is still not measured

One real YARG version, on one machine, against a corpus we wrote ourselves. The corpus is adversarial by design, but it is still ours: it cannot contain the case nobody thought of. A larger body of real community charts would test different things, and the licensing constraints on where those may come from are in the section above.